



NCRD's Sterling Institute of Management Studies
(NAAC Accredited A+ Grade)

Nerul, Navi Mumbai

2025-2027

GENERATIVE AI 10-Day Workshop

Project Documentation

On

**AI Chatbot Integration using OpenRouter API And
Gemini API**

Under the Guidance

of

Mr. Navneet Sir

By

Name : Shubham Terse

Roll No : 122

SYMCA Div:B

Title

Building an LLM-Powered AI Chatbot using Python and the OpenRouter API and Gemini API

Aim

To understand the fundamentals of Generative AI by developing and configuring a Python-based chatbot that interacts with Large Language Models (LLMs) via an API proxy network.

Objectives

- To explore Generative AI fundamentals and understand how LLMs generate text.
- To comprehend core concepts like tokens, embeddings, and context window management.
- To implement a working Python application capable of sending inference requests to cloud-hosted open-weights models (such as Llama 3.2 and Qwen 2.5).
- To securely manage API credentials within a Google Colab environment.

Theory & Background Knowledge

Before analyzing the code, it is essential to understand the underlying Generative AI framework that powers the chatbot:

- **Generative AI & LLMs:** Generative AI refers to systems capable of generating new content (text, code, images). Large Language Models (LLMs) are a subset of this, specifically trained on vast datasets of text to understand and generate human-like language based on statistical probability.
- **Transformer Architecture (High-Level):** Modern LLMs rely on the Transformer architecture. Instead of reading text sequentially, Transformers use a mechanism called *self-attention* to weigh the importance of all words in a sentence simultaneously, allowing the model to capture deep semantic context.
- **Tokens and Tokenization:** LLMs do not read words; they read "tokens." Tokenization is the process of breaking down text into smaller sub-word units.
- **Context Window:** The context window is the maximum number of tokens an LLM can process in a single request (both the input prompt and the generated output combined). In our implementation, utilizing the openrouter/free endpoint provides access to models with large context windows, allowing for extensive conversation histories.
- **Parameters, Training, and Inference:** A model's "parameters" are the billions of adjustable weights it learned during its initial *Pre-training* phase. Our Python code executes the *Inference* phase—where the trained model processes our prompt and generates a response in real-time.
- **Open-Weights Models (Llama 3.2 / Qwen 2.5):** Rather than relying solely on proprietary models, the OpenRouter API allows us to dynamically access top-tier open-source models

like Meta's Llama 3.2 and Alibaba's Qwen 2.5. These models have undergone rigorous *Instruction Tuning* to respond safely and accurately to user commands.

Overview

This module demonstrates how to implement an AI chatbot within a Google Colab environment. Rather than binding directly to a single provider, this implementation leverages the official OpenAI Python SDK and redirects it to the **OpenRouter API**. OpenRouter acts as a unified proxy that normalizes the schema across models and providers. By routing requests through OpenRouter, developers gain access to over 500 models (including free-tier inference models) without having to rewrite their core integration logic.

Prerequisites & Environment Setup

To execute this script successfully, the environment must be correctly configured to handle secure credentials.

Required Accounts

- **Google Account:** To execute the notebook via Google Colab.
- **OpenRouter Account:** To generate an API key (available at openrouter.ai).

Secure Key Management

Hardcoding API keys is a security risk. This project utilizes Google Colab's native Secrets manager:

1. Navigate to the ☐ **Secrets** panel on the left sidebar of your Colab interface.
2. Click **Add new secret**.
3. Name the secret exactly: GenAi_Chatbot.
4. Paste your OpenRouter API key into the **Value** field.
5. Toggle the **Notebook access** switch to allow the runtime to read the key.

Building an LLM-Powered AI Chatbot using Python and the OpenRouter API

Code:

```
!pip install -q -U openai
from openai import OpenAI
from google.colab import userdata
# Fetch your OpenRouter key from Colab secrets
key = userdata.get("Openai")
# Initialize client pointing to OpenRouter
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=key
)
prompt_text = "Do you know about Clash of Clans"
response = client.chat.completions.create(
    model="openrouter/free",
    messages=[
        {"role": "user", "content": prompt_text}
    ]
)
# Print which specific model was selected by the free router
print(f"Model used: {response.model}")
print("-" * 40)
# Print the result
print(response.choices[0].message.content)
```

Output :

```
[2]
✓ 15s !pip install -q -U openai

from openai import OpenAI
from google.colab import userdata

# Fetch your OpenRouter key from Colab secrets
key = userdata.get("Openai")

# Initialize client pointing to OpenRouter
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=key
)

prompt_text = "Do you know about Clash of Clans"

response = client.chat.completions.create(
    model="openrouter/free",
    messages=[
        {"role": "user", "content": prompt_text}
    ]
)

# Print which specific model was selected by the free router
print(f"Model used: {response.model}")
print("-" * 40)

# Print the result
print(response.choices[0].message.content)
```

... Model used: inclusionai/ling-3.0-flash-fin:free

Yes, I know a lot about **Clash of Clans**! It's one of the most popular and successful mobile strategy games ever made. Here's a quick overview:

- **Developer:** Supercell (a Finnish game studio)
- **Released:** August 2012 (iOS), with Android following shortly after
- **Genre:** Strategy / Base-building / MMO
- **Platforms:** iOS and Android
- **Business Model:** Free-to-play with in-app purchases

How It Works:

- **Build & Upgrade:** Players construct and upgrade a village with resource collectors, defenses, barracks, and other buildings.
- **Train Troops:** You train armies consisting of various troop types (Barbarians, Archers, Dragons, Giants, Goblins, etc.) and spells (Rage, Freeze, Lightning, etc.).
- **Attack & Defend:** You defend your own village and attack other players' villages to earn **gold**, **elixir**, and **dark elixir**.
- **Heroes:** Unique characters like the **Barbarian King**, **Archer Queen**, **Grand Warden**, and **Royal Champion** that level up alongside your village.
- **Clans:** Join or create clans (up to 50 players) to participate in **Clan Wars**, **Clan Games**, and the **Clan Capital**.
- **Town Hall Levels:** Progression through Town Hall levels (currently up to Town Hall 16+) unlocks new buildings, troops, and mechanics.
- **Builder Base & Battle Base:** Alternative gameplay modes separate from the main village.

Key Highlights:

- One of the **highest-grossing mobile games** of all time, generating billions in revenue.
- Active **esports scene**, including the annual **Clash of Clans World Championship**.
- Supercell releases regular updates with balance changes, new content, and seasonal events.

Whether you're a longtime player or just curious, feel free to ask me anything specific about Clash of Clans – strategies, troop compositions, base building tips, clan mechanics, you name it!

Building an LLM-Powered AI Chatbot using Python and the Gemini API**Code:**

```
# 1. Install the correct new package
!pip install -q -U google-genai langchain
# 2. Use the new import statement
from google import genai
from google.colab import userdata
# Fetch your API key from Google Colab's secrets manager
key = userdata.get("Gemini_API_Key")
# 3. Initialize the Client (Configuration has changed in the new SDK)
client = genai.Client(api_key=key)
# Define your prompt
prompt_text = "give me strategies of clash of clans to attack on th14"
# 4. Generate the response using the client.models method
response = client.models.generate_content(
    model='gemini-3.5-flash',
    contents=prompt_text
)
# Print the result
print(response.text)
```

Output:

```
[ ]  # 1. Install the correct new package
!pip install -q -U google-genai langchain

# 2. Use the new import statement
from google import genai
from google.colab import userdata

# Fetch your API key from Google Colab's secrets manager
key = userdata.get("Gemini_API_Key")

# 3. Initialize the Client (Configuration has changed in the new SDK)
client = genai.Client(api_key=key)

# Define your prompt
prompt_text = "give me strategies of clash of clans to attack on th14"

# 4. Generate the response using the client.models method
response = client.models.generate_content(
    model='gemini-3.5-flash',
    contents=prompt_text
)

# Print the result
print(response.text)
```

```
... WARNING:google_genai.models.Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct
At Town Hall 14 (TH14), the meta introduces **Hero Pets** and the devastating **Giga Inferno Poison Bomb** (the Town Hall drops poison when destroyed). Weaponized Builder Huts also heal surroundi

To 3-star consistently, you need strategies that can handle the Town Hall poison and gut the core of the base. Here are the top 4 attack strategies for TH14, ranging from spam-friendly to high-sk

---

#### 1. Super Archer Blimp + Hydra (Dragons & Dragon Riders)
*This is currently one of the easiest and most consistent 3-star strategies at TH14. It relies on the "Sarch" Blimp to clear 50% of the base (including the TH) before the main army is even deploy

* **Army Composition:** 5-6 Dragons, 4-5 Dragon Riders, 4-6 Balloons, 1 Baby Dragon (for funneling).
* **Spells:** 1 Rage, 5 Invisibility, 2 Clone (or 1 Clone + 2 Freeze).
* **Siege Machine:** Battle Blimp (filled with 3 Super Archers, 1 Super Goblin, 1 Wall Breaker).
* **Hero Pets:** King (Yak), Queen (Unicorn), Warden (Owl), Royal Champion (Lassi).
* **Hero Equipment:** King (Giant Gauntlet + Rage Vial), Queen (Frozen Arrow + Invis Vial), Warden (Eternal Tome + Healing Tome).

#### **How to Execute:**
1. **The Blimp Drop:** Deploy a Lava Hound or a few Balloons to tank, then send the Battle Blimp towards the core of the base (ideally near the Town Hall, Eagle Artillery, or Scattershots).
2. **The Clone/Invis Trick:** Use the Grand Warden's ability early if needed to protect the Blimp. Pop the Blimp manually over a high-value compartment. Immediately drop 1 Invisibility Spell, 1
3. **The Invisibility Cycle:** Keep the Super Archers invisible by dropping another Invisibility Spell every **4 seconds** (do not let them become visible, but do not cover the defenses in the s
4. **The Hydra Clean-up:** Use your King and Queen on one side to create a funnel. Send the Dragons, Dragon Riders, and Grand Warden down the middle. Use the Royal Champion to clear leftover out

---

#### 2. Queen Charge Hybrid (Miner + Hog Rider)
*A classic, highly reliable strategy. The Queen Charge takes out the Town Hall or key defenses, while the Hybrid cleans up the rest.*

* **Army Composition:** 5 Healers, 10-12 Miners, 10-12 Hog Riders, 2-3 Super Wall Breakers, 2 Baby Dragons (funneling), 1-2 Headhunters.
* **Spells:** 3 Rage, 2 Heal, 1 Freeze, 1 Poison.
* **Siege Machine:** Siege Barracks (filled with Hog Riders) or Flame Flinger.
* **Hero Pets:** Queen (Unicorn), King (Yak), Warden (Owl), RC (Lassi).

#### **How to Execute:**
1. **The Queen Charge (QC):** Start your Queen and Healers. Use Baby Dragons/Sneaky Goblins to funnel her into the base. Your goal is to take out the **Town Hall** (to avoid the poison hitting y
2. **The Funnel:** Deploy your King and the Siege Barracks on the opposite side of the Queen Charge to create a funnel. The Wizards and P.E.K.K.A from the Siege Barracks will clear the outer tra
3. **The Hybrid Deploy:** Send the Miners first, followed closely by the Hog Riders and Grand Warden, right down the center of the funnel created by the Queen and King.
4. **Spells & Abilities:** Use your Heal Spells when the Hybrid hits Giant Bombs, Scattershots, or Multi-Infernos. Use the Warden's Eternal Tome when they face high burst damage. Send the Royal

---

#### 3. Super Bowler Smash
*Super Bowlers do insane bounce damage. When funneled correctly, they can tear through a TH14 base in seconds.*

* **Army Composition:** 4-5 Super Bowlers, 5 Healers, 1 Ice Golem, 1 Electro Titan (or P.E.K.K.A), 3 Super Wall Breakers, 4-5 Balloons.
* **Spells:** 2 Rage, 2 Jump, 1 Healing, 2 Freeze, 1 Poison.
* **Siege Machine:** Flame Flinger (to clear a corner) or Log Launcher (to open up the base).
* **Hero Pets:** Queen (Unicorn), King (Yak), Warden (Healing Tome + Eternal Tome), RC (Lassi).

#### **How to Execute:**
1. **The Warden Walk / Flame Flinger:** Start a Warden Walk (Warden + Healers) on one corner of the base to clear it. On the adjacent corner, drop your Flame Flinger or King to clear the other s
2. **The Entry:** Pull your Warden over to your main army once the funnel is made. Drop your Ice Golem, Super Bowlers, Queen, and Log Launcher/King heading directly into the core.
3. **Rage & Jump:** Jump them into the core. Drop a Rage Spell and use the Warden's Ability as they enter the core to protect them from the Town Hall poison and Eagle Artillery shots.
4. **Bouncing Destruction:** The Super Bowlers will bounce their rocks and destroy the core from a safe distance. Use the Royal Champion to clean up defenses on the backside.
```

Conclusion & Key Takeaways

Building this AI chatbot bridges the gap between theoretical Generative AI concepts and practical application. By implementing this code, we successfully applied several core topics from the syllabus in a real-world scenario:

- **Generative AI Fundamentals & LLMs:** We moved from theory to practice by directly interacting with a Large Language Model to generate text dynamically based on a user prompt.
- **Parameters, Training, and Inference:** Through the OpenRouter API, we executed an active inference request. The model processed our input prompt (context) and generated a response, demonstrating the end result of model training and text generation in real time.
- **Llama 3.2 & Qwen 2.5 Ecosystem:** By utilizing OpenRouter's openrouter/free endpoint, we tapped into the same unified API ecosystem that hosts state-of-the-art open-weights models like Llama 3.2 and Qwen 2.5, demonstrating how modern open-source models are accessed programmatically.
- **Python for GenAI Applications:** We utilized Python as the primary application layer to securely manage credentials, structure the API payload using the standard openai library, and parse the complex JSON response.

Ultimately, this project serves as the foundational stepping stone. By successfully establishing an API connection and parsing an LLM's output, the groundwork is laid for advancing to more complex orchestration frameworks like **LangChain**, running **Local LLMs via Ollama**, and building full-scale AI agents.